

Crafting Interpreters VM

L0 Essence: The essence of compilers and virtual machines is to translate high-level code into bytecode instruction streams (OPCODE) via lexical and syntactic analysis (Pratt parsing), driven by a stack-based computing model and local call frame execution, and managed by a tri-color garbage collector (GC) that automates heap allocation life cycles.

[M1.1] Lexical Scanner: Regex State Transitions and Tokenization

The Scanner reads source code character-by-character, using start and current pointers to track Token physical spans. It implements a state machine (via switch-case) to parse single/double character operators (e.g., !=), resolves identifiers against a reserved keyword hash map, and outputs a list of Tokens annotated with type, lexeme, and line number.

[M1.2] Recursive Descent Parsing: Eliminating Left Recursion for LL(1) Grammars

The syntactic analyzer parses CFG grammar rules starting from the lowest precedence logical assignments. Left recursion is refactored into iterative loops to prevent infinite recursions, resolving operator precedence hierarchies and ensuring a single-pass LL(1) recursive descent parser without backtracks.

[M1.3] Abstract Syntax Tree (AST) Design and the Visitor Pattern

The parser outputs an AST to represent syntactic relationships. The AST base class defines an accept(Visitor) method, implemented by sub-classes (Binary, Unary, Grouping) to dispatch execution. This Visitor pattern decouples the syntactic tree structure from compiler optimization and interpreter evaluation logic.

[M1.4] Dynamic Scope Environment: Variable Binding and Scoped Resolution

The execution scope environment (Environment) maps variable identifiers to runtime values via a hash map. For nested scopes, the new child Environment holds an enclosing pointer to its parent. During resolution, the interpreter walks up the parent pointers to locate variables in the correct lexical scope.

[M1.5] Tree-Walk Interpreter: Execution Limitations and Performance Bottlenecks

The jlox interpreter evaluates code by traversing the AST recursively and executing visitor methods on each node. Because every mathematical operation or variable resolution triggers polymorphic method dispatch and instantiates temporary host objects (Java/C

[M2.1] Bytecode Instruction Set (OPCODE) and Chunk Storage Layout

To achieve high performance, clox compiles source files into linear bytecode. Each fundamental operation is represented as a single-byte Opcode (e.g., OP_ADD). The instruction array, a constant pool (ValueArray), and compressed line-number arrays are packed into a Chunk structure for fast cache-friendly execution.

[M2.2] Stack-Based VM Model: Instruction Decoding and Interpreter Loop

clox implements a stack-based VM model, maintaining an Operand Stack as a continuous array of Values. The Instruction Pointer (IP) references the active bytecode location. The interpreter loop reads instructions via a fast for(;;) loop and a READ_BYTE() macro, dispatching stack push and pop instructions through a switch statement.

[M2.3] Runtime Value Representation: Tagged Unions and NaN Tagging

VM values are represented as tagged unions. Advanced optimization utilizes NaN Tagging: under the IEEE 754 double-precision standard, NaN values are defined by exponent bits set to all 1s and a non-zero mantissa. NaN tagging exploits the remaining 51 unused payload bits to pack pointers, booleans, and nil values into a single 8-byte value.

[M2.4] VM Low-Level Diagnostics: Disassembler and Stack Tracing

The disassembler translates raw bytecode into readable assembly instructions (e.g., OP_POP). In debug mode, prior to executing each opcode in the loop, the VM disassembles the instruction at the current IP and logs the operands currently active on the stack to assist debugging.

[M3.1] Pratt Parsing: Binding Power and Operator Associativity

Pratt Parsing is an elegant, table-driven expression parsing technique. Each token type is associated with a binding power (Precedence). The parser compares the precedence of subsequent tokens against the current binding power, looping to consume tokens of higher precedence to handle binary operator associativity.

[M3.2] Prefix and Infix Parse Rule Mappings

The parser rules table maps tokens to two parsing function pointers: prefix functions compile unary operators, variables, and grouping parentheses; infix functions compile binary infix operators and function calls. This rule table replaces deeply nested switch-case functions with table-driven dispatches.

[M3.3] Precedence Table Hierarchy and Expression Routing

Precedence levels increase from PREC_NONE (lowest) → PREC_ASSIGNMENT → PREC_COMPARISON → PREC_TERM (addition/subtraction) → PREC_FACTOR (multiplication/division) → PREC_UNARY → PREC_PRIMARY. The compiler invokes parsePrecedence(precedence) to parse expressions above the specified threshold.

[M3.4] Pratt Precedence Compilation: Parsing '1 + 2 * 3'

When compilation starts at parsePrecedence(PREC_ASSIGNMENT):

1. The parser consumes 1, calling the prefix rule number() which emits OP_CONSTANT 1.
2. It encounters + (precedence PREC_TERM). Since PREC_TERM is higher than PREC_ASSIGNMENT, it calls the infix rule binary(), passing PREC_TERM recursively.
3. It parses 2 as prefix. The next operator * (PREC_FACTOR > PREC_TERM) forces the loop to continue, calling binary() recursively for 3 and emitting OP_MULTIPLY, then returning to emit OP_ADD.

L1 Logical Framework

- 1. Lexical Parsing & AST Modeling:** The Scanner converts source text into a Token stream, feeding a recursive descent parser to construct an Abstract Syntax Tree (AST) that utilizes the Visitor pattern to decouple syntax structures from Interpreter execution.
- 2. Bytecode Chunking & VM Execution:** Expressions are compiled into a linear instruction array (Chunk) using Pratt Parsing, which is executed by a switch-loop interpreter loop that operates on an Operand Stack with an Instruction Pointer (IP).
- 3. Frame Stacking & Upvalue Closures:** Nested scopes and closures are resolved by packaging variables into Upvalue structures that migrate from the VM stack (Open) to the heap (Closed) as call frames pop, bypassing stack life cycle bounds.
- 4. Tri-Color Marking & Sweeping GC:** Memory is dynamically collected by a tri-color GC triggered by heap allocation doubling, tracking alive pointers via a Gray Stack to blacken reachable objects, removing weak references, and sweeping remaining white objects.

L2 Data Flow Topology

• Source String → Scanner Token Stream → Pratt Compile → Bytecode
Chunk → VM Run ip++ → READ_INSTRUCTION → Switch OPCODE → Push/Pop
Operand Stack → Heap Allocation → Triggers GC (allocated > nextGC)
→ Mark Roots to Gray → Pop Gray & Push Children → Blacken active
objects → Sweep remaining white objects → Free Memory.

Trade-off Matrix: Compiler & Execution

▮ Execution Model vs Performance: Tree-Walk Interpreter (jlox) → Bytecode Virtual Machine (clox) [jlox traverses the AST using the Visitor pattern, making it highly readable and easy to debug → but polymorphic dispatch and host memory allocations make it extremely slow with a high memory footprint [sacrifices speed to simplify implementation]]

▮ Value Representation: Tagged Unions → NaN Tagging [Tagged unions are portable and easy to extend → but NaN tagging compresses values into 8 bytes, maximizing CPU L1/L2 cache locality and memory density [improves cache layout, slightly raises bitwise parsing costs]]

▮ Parsing vs Compiler Size: Recursive Descent Parser → Pratt Parsing [Recursive descent parsers are intuitive → but Pratt Parsing uses a table-driven rules layout that collapses complex operator precedences into a single loop, minimizing parser source code size [reduces code lines, slightly steeper learning curve]]

▮ GC Strategy vs STW Latency: Tri-Color Mark-Sweep GC → Reference Counting [Mark-Sweep GC handles cyclic structures easily with zero allocation overhead → but requires Stop-The-World (STW) pauses during collection. Real-time VM workloads require concurrency barriers [eliminates read/write barriers, introduces collection pauses]]

[M4.1] Heap Memory Management and Dynamic Type Tagging

All heap-allocated objects (Obj) share a common header containing a garbage collection mark bit and a type tag (e.g., ObjString, ObjClosure). The VM tracks these via a global linked list of heap objects, casting them to target struct types at runtime based on the tag.

[M4.2] Hash Table Design: Open Addressing and Linear Probing

clox implements a hash table using open addressing with linear probing: `index = (hash + i) % capacity`. Key-value pairs are stored directly in a contiguous array to maximize CPU cache locality. When the load factor exceeds 75%, the table doubles in capacity and rehashes all active entries.

[M4.3] Hash Table Deletions: Tombstone Fencing

During deletion, entries cannot be simply cleared as this would break linear probing chains. Instead, the deleted slot is marked as a Tombstone. Hash lookups treat tombstones as active to continue probing, while insert operations reuse tombstone slots to prevent unbounded array growth.

[M4.4] String Interning: Single-Instance Pointer Comparison

The VM maintains a global hash table for String Interning. Before allocating a new ObjString, the heap allocator queries the intern table. If a matching string exists, the VM reuses its pointer; otherwise, it allocates the string and registers it. This reduces string equality checks from $O(N)$ to $O(1)$ pointer comparison.

[M5.1] Branching and Loops: Backpatching Bytecode Offsets

When compiling conditional branches (e.g., `if`), the compiler emits a jump instruction (`OP_JUMP`) with a two-byte placeholder offset. It then compiles the body, calculates the actual byte offset of the compiled statements, and "backpatches" the distance into the placeholder.

[M5.2] Closures: Upvalues and Lexical Scope Capture

Functions referencing variables outside their local scope are packaged into an `ObjClosure` with an array of Upvalues. The compiler identifies these free variables and generates Upvalue indices, allowing the closure to reference these variables at runtime.

[M5.3] Upvalue Lifecycle: Open and Closed Upvalues

- **Open Upvalue:** While the captured variable resides in an active stack frame, the Upvalue's location pointer references the target variable directly on the VM stack.
- **Closed Upvalue:** When the capturing frame returns, the VM moves the variable from the stack into the Upvalue structure's heap memory and updates the location pointer to reference its own payload.

[M5.4] Object-Oriented Programming: Classes, Instances, and Methods

An `ObjClass` contains a hash map of methods. Instantiation creates an `ObjInstance` pointing to its class, which maintains a local hash table of fields. When a method is called, the VM retrieves it from the class methods table and packages it with the receiver (`this`) into an `ObjBoundMethod` on the stack.

[M5.5] Class Inheritance and Method Bindings

During class compilation, the subclass copies the methods table of its superclass. When compiling `super.method()`, the compiler resolves the superclass statically and binds it to the active `this` receiver at runtime, ensuring correct method overrides and dispatch.

[M6.1] GC Roots and the Allocation Threshold Formula

The GC uses an allocation threshold trigger: `allocatedBytes > nextGC`, where `nextGC` doubles the current memory. When triggered, the GC marks all roots—the VM stack, active CallFrames, global variables, and active compiler definitions—as gray.

[M6.2] Tri-Color Marking: White, Gray, and Black Objects

- **White:** Unmarked objects eligible for reclamation. All objects start white at GC initialization.
- **Gray:** Objects reachable from roots but whose nested references have not yet been scanned.
- **Black:** Reachable objects whose children have been fully scanned and colored gray.

[M6.3] Gray Stack Worklist and Overflow Fencing

The GC maintains a worklist array (Gray Stack) to store references to gray objects. If extreme memory depletion causes the gray stack to fail reallocation, the GC falls back to a fail-safe mechanism: it marks all objects as gray and traverses the heap list slowly.

[M6.4] Reference Tracing and Blacken Transitions

The GC loops while the Gray Stack is not empty, popping the top gray object and marking it black (Blacken). It then traverses the object's children; any white child is colored gray and pushed onto the Gray Stack to propagate the marking sweep.

[M6.5] Weak Reference Removal: String Intern Table Pruning

The global string intern table contains weak references. To prevent memory leaks, after the marking phase ends but before sweeping starts, the GC scans the intern table and prunes any keys pointing to white (unmarked) strings.

[M6.6] Sweep Phase: Reclaiming Memory and Heap List Pruning

The sweep phase traverses the global linked list of heap objects. If an object remains white, the GC unlinks it from the heap list and invokes `freeObject()` to reclaim its physical C memory; if it is black, it is reset to white and its mark cleared for the next cycle.

Zone T1: Opcode & Parameters

`OP_CONSTANT` : Pushes a constant from the constant pool onto the operand stack.

`\OP_GET_LOCAL` / `OP_SET_LOCAL` : Reads or modifies a local variable slot relative to the stack frame pointer. `\OP_JUMP` / `OP_LOOP` : Performs conditional/unconditional forward or backward jumps by patching the IP register. `\OP_CLOSURE` : Instantiates a function template, capturing active variables into Upvalues. `\OP_INVOKE` : Optimizes method calls by looking up the class methods map and executing the function in a single instruction. `\GC_HEAP_GROW_FACTOR` = 2

: Memory threshold growth multiplier triggered after each garbage collection run.

Zone T2: VM Diagnostics & Debugging